

CSC311: Introduction to Machine Learning

Project Report

What's on the Canvas? Classifying Paintings from Survey Responses

Submitted by

Group 46915

Qinyue (Alex) Li, Zhaotong (Macqueen) Pan, Qian (Angela) Su

University of Toronto
Winter 2026

1 Executive Summary

We explored three model families for classifying survey responses into one of three paintings: K-Nearest Neighbors (KNN), Softmax Regression, and a Multi-Layer Perceptron (MLP). All models were trained on the same 14-feature set (numeric, Likert, multi-hot categorical, and TF-IDF text) and tuned via stratified GroupKFold 5-fold cross-validation with macro-F1 as the selection criterion. Our best-performing model is Softmax Regression (LBFGS, $C=1$), achieving a projected test accuracy of **89%**. Softmax outperformed the MLP (88.2%) and KNN (85.8%) on the held-out test set, likely because the approximately linear class separation in our feature space favours a simpler model with fewer parameters to overfit, while KNN's distance-based approach struggles in the 172-dimensional feature space.

2 Data Exploration

2.1 Dataset Overview

The target variable is Painting, which consists of three classes, each containing 562 observations (1686 total), resulting in a perfectly balanced class distribution. The dataset includes 14 input features of four types: 4 continuous numerical features (emotion intensity, number of objects noticed, number of prominent colours, willingness to pay), 4 ordinal categorical features measured on 5-point Likert scales (e.g., agreement with emotional statements), 3 nominal categorical features (preferred room, viewing companion, associated season), and 3 free-text response features (e.g., imagined soundtrack, painting description). Most numeric features are roughly unimodal with moderate spread, except willingness to pay, which is heavily right-skewed with values spanning from 0 to 10^{30} due to joke responses. Likert features show class-dependent response patterns (e.g., Figure 2).

2.2 Issues

- Several features contain a small number of missing values, typically between 65 and 85 per feature (approximately 4–5 % of the dataset).
- Existence of outliers. For example, an extreme value (100) is observed in the feature “How many prominent colours do you notice in this painting?”. For most features, we retain outliers: median imputation and z-score normalization already reduce their influence on model training, and removing them risks discarding legitimate responses. The willingness-to-pay feature is a special case: raw values span 30 orders of magnitude (from 0 to 10^{30}) due to joke responses, causing the standard deviation to be $\sim 2.6 \times 10^{28}$. After plain z-score normalization, all reasonable values collapse to approximately the same score, destroying the feature's discriminative power. We address this with a $\log(1 + x)$ transform (see Pre-processing).
- Inconsistencies. For features with mixed data types, some entries are numerical values while others include symbols or non-numeric text.

2.3 Pre-processing

- Missing value imputation. Numeric and Likert features are imputed using the median to reduce sensitivity to skewness and outliers. Missing nominal categorical entries are

represented as all-zero multi-hot vectors (no category selected), which is the natural default for multi-select fields where the respondent may not have answered. (Our proposal planned most-frequent-response imputation for categorical features; we switched to all-zero vectors because multi-hot fields have no single “most frequent” category.)

- Normalization. Numeric and Likert features are normalized to zero mean and unit variance to prevent large-range features from dominating distance calculations. Likert responses (e.g., “4 - Agree”) are first parsed to their integer values (1–5) and then treated as numeric for imputation and normalization.
- Type consistency. The feature “How much (in Canadian dollars) would you be willing to pay for this painting?” is cleaned by removing currency symbols and non-numeric texts, then converting all entries to numerical values.
- Outlier-robust pay transformation. After cleaning, negative pay values are clipped to 0, and the column is transformed via $\log(1 + x)$ before z-score normalization. This compresses the 30-order-of-magnitude range into $[0, 69]$, producing a standard deviation of ~ 4.5 and preserving the feature’s ability to distinguish between classes. The $\log(1 + x)$ function is parameter-free, so no training-set statistics are used in the transform itself; only the subsequent z-score statistics are fitted on training data.
- Multi-hot encoding. Nominal categorical features (room, companion, season) allow multiple selections separated by commas. Each unique category becomes a binary indicator column, producing a multi-hot representation.
- Text features. Missing text entries are treated as empty strings, yielding all-zero TF-IDF vectors. Text features are then transformed using TF-IDF, which converts text into numerical vectors while down-weighting common words.

2.4 Data splitting

All exploratory analysis, visualizations, and statistics reported in this section were computed exclusively on the training set to prevent information from the test set from influencing modelling decisions.

The dataset is divided into training and test sets, with 80% of the data allocated to the training set and 20% to the test set. Since each student contributes three related observations, group-based splitting (GroupShuffleSplit by student ID) is applied to ensure that all responses from the same student remain in the same partition, preventing cross-set data leakage. Hyperparameter selection is performed using stratified GroupKFold 5-fold cross-validation on the training set, where fold assignment also respects student groups so that no student’s responses appear in both a training fold and a validation fold. All pre-processing transformations are fitted using only the training data (or training fold) and subsequently applied to the held-out set.

2.5 Key insights about predictive features

“How many objects caught your eye in the painting?” is a continuous feature. The boxplot (Figure 1) shows clear separation in medians and difference in spread across the three classes. “This art piece makes me feel sombre” is an ordinal categorical feature measured on a 5-point Likert scale (1 = Strongly disagree, 5 = Strongly agree). The stacked bar chart (Figure 2) shows distinct response distributions across the three classes.

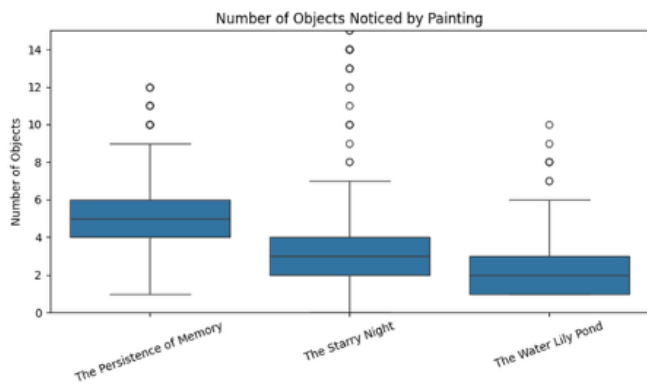


Figure 1: Number of objects noticed in each painting

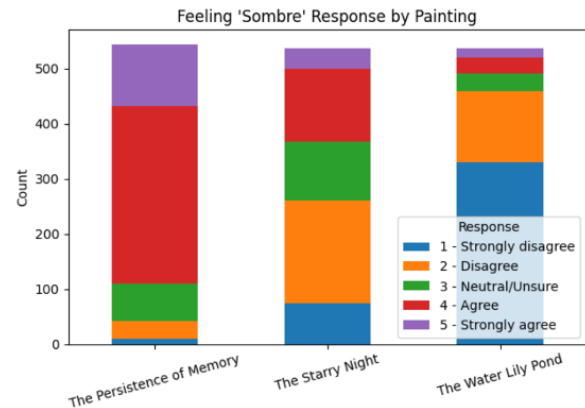


Figure 2: Feeling sombre response for each painting

2.6 Model choices

The approximately monotonic class separation in key features (Figures 1–2) suggests that a linear model such as **Softmax Regression** may already perform well. However, interactions across the four feature modalities (numeric, Likert, categorical, text) may create nonlinear boundaries, motivating an **MLP**. Finally, the clear clustering visible in Figure 1 suggests that distance-based **KNN** can exploit local structure, while also serving as a non-parametric baseline to verify that the parametric models learn something beyond nearest-neighbour similarity.

3 Methodology

All three models share the same validation protocol: stratified GroupKFold 5-fold cross-validation on the training set, where fold assignment respects student groups to prevent within-CV data leakage. All preprocessing transformations are fitted on the training fold and applied to the validation fold.

3.1 Softmax Regression

Reason: Softmax regression [1] is a natural choice for multi-class classification when the features show approximately linear separation across classes, as observed in our data exploration. The model is simple, interpretable, and robust on small datasets. It uses all 14 input features (numeric, Likert, categorical, and TF-IDF text) to leverage the full feature set.

Optimization Techniques:

- **Optimizer:** LBFGS and SGD. We compare both to determine which optimizer works better on our feature set.
- **Regularization:** L2 regularization
- **Early Stopping Strategies:** For LBFGS, early stopping is not required since the softmax objective is convex and LBFGS converges to a global minimum. For SGD, we set `max_iter=1000`.

Hyperparameters and Rationale:

- LBFGS: L2 regularization strength $C = 1/\lambda \in \{0.1, 1, 10\}$ and convergence tolerance $\text{tol} \in \{10^{-3}, 10^{-4}, 10^{-5}\}$. Max iterations fixed at 2000 as a safeguard.
- SGD: L2 regularization $\alpha \in \{0.001, 0.0001, 0.00001\}$, learning rate schedule $\in \{\text{optimal}, \text{constant}, \text{adaptive}\}$, initial learning rate $\eta_0 \in \{0.1, 0.01, 0.001\}$.

Hyperparameter tuning process: Grid search over 9 LBFGS and 21 SGD configurations, evaluated using macro-F1 on each validation fold. The best optimizer and its hyperparameters are selected jointly.

Deviation from proposal: Our proposal planned SGD with step or exponential decay schedules. We implemented Softmax Regression using `sklearn.LogisticRegression` (LBFGS) and `SGDClassifier`, which provide optimal, constant, and adaptive learning rate schedules instead. These serve the same purpose of controlling step size during training and are the natural choices within the `sklearn` framework.

3.2 Multi-Layer Perceptrons(MLP)

Reason: Multi-layer perceptrons [1, 2] are suitable for this task because the dataset contains both continuous features (emotion intensity, number of objects, number of colors) and high-dimensional categorical/text features (room preference, feelings, soundtrack descriptions, etc., after one-hot encoding or TF-IDF), which together form a nonlinear feature space.

Optimization Techniques:

- **Optimizer:** Adam (via `sklearn.MLPClassifier`)
- **Regularization:** L2 regularization (weight decay parameter α)
- **Early Stopping:** Monitored validation loss; stopped training when no improvement was observed for 5 consecutive epochs ($\text{patience}=5$), with $\text{max_iter}=200$ as an upper bound.

Deviations from proposal: Our proposal planned to test dropout $\in \{0.0, 0.1, 0.2\}$, custom learning rate schedules (cosine and step decay), and number of epochs $\in \{20, 30, 40, 60\}$. We implemented the MLP using `sklearn.MLPClassifier`, which does not support dropout or custom LR schedules. Instead, we rely on L2 regularization and early stopping for overfitting control, and Adam’s built-in adaptive learning rate. This is a reasonable substitute: L2 regularization penalizes large weights similarly to dropout’s noise injection, and Adam already adapts per-parameter step sizes, reducing the need for an explicit schedule. Removing dropout, LR schedule, and epoch count from the grid also reduced the search space from the proposal’s ~ 1400 configurations to 120, addressing TA feedback that the original grid was impractically large for 5-fold CV.

Hyperparameters and Rationale:

- Hidden layer architectures $\in \{(32), (64), (128), (64, 32), (128, 64)\}$: single-layer and two-layer configurations with ~ 1350 training samples. Deeper networks would likely overfit.
- Learning rate $\in \{0.01, 0.001\}$: Adam’s adaptive step sizes work well in this range.
- L2 regularization $\lambda \in \{0.0001, 0.001, 0.01\}$: spans light to moderate weight decay to control overfitting without underfitting.

- Activation function $\in \{\text{ReLU}, \tanh\}$: ReLU is the default for efficient gradient flow; tanh can work better on normalized inputs.
- Batch size $\in \{32, 64\}$: smaller batches add noise that can help regularization; both are reasonable for this dataset size.

Hyperparameter tuning process: Grid search over all 120 combinations, evaluated using macro-F1 on each validation fold.

3.3 K-Nearest Neighbors (KNN)

Reason: KNN is a non-parametric baseline that classifies by majority vote among the k nearest training points [2], making no assumptions about decision boundary shape. Our 172-dimensional feature space (~ 1350 samples) risks the curse of dimensionality [1], but we include KNN to test whether the clear per-class clustering observed in data exploration overcomes this limitation.

Optimization Techniques:

- **Optimizer:** Not applicable. KNN has no trainable parameters and therefore requires no optimization procedure.
- **Regularization:** The value of k controls the bias–variance tradeoff: a small k gives more flexible (but noisier) predictions, while a large k smooths things out at the cost of potentially missing local structure. We search over a range of k values to find the right balance.

Hyperparameters and Rationale:

- Number of neighbours $k \in \{3, 5, 7, 11, 15, 21\}$: odd values avoid ties in majority voting. The range covers the typical bias–variance tradeoff from highly local ($k=3$, 0.2% of training samples) to smoothed ($k=21$, 1.6%).
- Distance metric $\in \{\text{Euclidean}, \text{Manhattan}\}$: Euclidean penalizes large deviations more heavily; Manhattan is more robust to occasional outliers.
- Weighting scheme $\in \{\text{uniform}, \text{distance}\}$: distance weighting lets closer neighbours have more influence, which can help when the class boundary falls within the neighbourhood.

Hyperparameter tuning process: Grid search over all 24 combinations, evaluated using macro-F1 on each validation fold.

3.4 Evaluation Metrics

We use three complementary metrics to evaluate and compare all models:

- **Accuracy** measures the overall proportion of correct predictions. Because our dataset is perfectly balanced (562 samples per class), accuracy is a meaningful metric that is not distorted by class imbalance.
- **Macro-F1 score** averages the F1 score across all three classes, where each class’s F1 is the harmonic mean of its precision and recall. This is useful because accuracy alone can look fine even if the model does poorly on one particular painting; macro-F1 would drop in

that case, making the problem visible. We use macro-F1 as the primary criterion when selecting hyperparameters during cross-validation.

- **Per-class precision and recall** are reported alongside macro-F1 to diagnose which classes, if any, are systematically misclassified.

3.5 Implementation

All three models were trained and tuned using scikit-learn (KNeighborsClassifier, LogisticRegression, SGDClassifier, MLPClassifier). Text features were vectorized with TfidfVectorizer. For the final prediction script (pred.py), we exported the trained Softmax weights, intercept, preprocessing statistics, and TF-IDF vocabularies, then reimplemented the full pipeline and softmax inference using only numpy and pandas, as required by the submission constraints.

4 Results

Table 1 compares the best configuration from each model family after grid search. KNN ($k=5$, Euclidean, distance weighting) achieves a CV macro-F1 of 0.808; its performance drops for $k \geq 11$, consistent with the bias–variance tradeoff. For Softmax, LBFGS ($C=1$, $\text{tol}=10^{-3}$) and the best SGD configuration ($\alpha=0.001$, adaptive, $\eta_0=0.1$) achieved nearly identical test macro-F1 (0.890 vs. 0.891); we selected LBFGS for its deterministic convergence. The best MLP (128-64, ReLU, $\text{lr}=0.01$, $\alpha=0.001$, batch size 64) achieved the highest CV macro-F1 of 0.890, but Softmax achieved the best test macro-F1 (0.890), suggesting the MLP’s additional capacity slightly overfits on this dataset size.

Table 1: Comparison of best models across all three families.

Model	CV Macro-F1	Train Acc	Test Acc	Test Macro-F1
KNN ($k=5$, Euclidean, distance)	0.808	97.5%	85.8%	0.858
Softmax (LBFGS, $C=1$, $\text{tol}=10^{-3}$)	0.873	92.2%	89.1%	0.890
MLP (128-64, ReLU, $\alpha=0.001$)	0.890	95.3%	88.2%	0.880

Overfitting analysis. The train–test accuracy gaps in Table 1 reveal different degrees of overfitting across models. KNN shows the largest gap (97.5% $\rightarrow$ 85.8%, $\Delta=11.7\%$), consistent with the curse of dimensionality [1]: in 172 dimensions, the neighbourhood around each point is sparse and training neighbours are unreliable proxies for test-time neighbours. The MLP’s gap ($\Delta=7.1\%$) is moderate, indicating that its extra capacity memorizes some training patterns that do not generalize. Softmax has the smallest gap ($\Delta=3.1\%$), confirming that the linear model generalizes best on this dataset.

Error analysis. Across all three models, *The Starry Night* is the hardest class: its test F1 is 0.79 (KNN), 0.84 (Softmax), and 0.83 (MLP). The Softmax confusion matrix (Figure 3) shows that Starry Night errors are nearly evenly split between Persistence of Memory (9 samples) and Water Lily Pond (10 samples), suggesting that Starry Night shares feature characteristics with both other classes. The overlap with Persistence of Memory likely stems from shared strong emotional responses (e.g., high sombre agreement, as shown in Figure 2). The Softmax model’s per-class test metrics are: Persistence 0.92/0.89/0.91, Starry Night 0.85/0.83/0.84, Water Lily Pond 0.90/0.95/0.92 (precision/recall/F1).

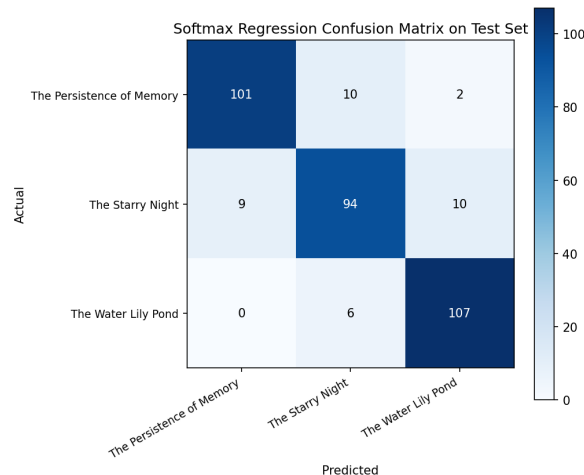


Figure 3: Softmax Regression confusion matrix on the held-out test set.

Final model and test performance estimate. We select Softmax Regression as our final model for the prediction script. We estimate its accuracy on the unseen test set to be **89%**. The CV macro-F1 (0.873) and held-out test macro-F1 (0.890) are close, indicating stable generalization with no sign of overfitting. Despite the MLP achieving a higher CV macro-F1 (0.890), its test macro-F1 (0.880) falls below Softmax's, consistent with the principle that simpler models can generalize better when the training set is small (~ 1350 samples) relative to the feature space (172 dimensions).

5 Contributions and Learning

- **Angela Su:** Implemented the KNN model (methodology, tuning, and evaluation), the evaluation metrics framework, and `pred.py`; wrote most of the final report. The most important lesson I learned is that the bias–variance tradeoff directly manifests in KNN through the choice of k : small k overfits to noisy neighbours while large k loses the local structure that makes KNN effective.
- **Macqueen Pan:** Implemented the softmax regression model, including hyperparameter tuning and final evaluation on the test set. Developed the preprocessing pipeline. The most important lesson I learned is that preprocessing must be consistent between the training set and unseen data to avoid data leakage and ensure fair evaluation.
- **Alex Li:** Implemented the MLP regression model, including hyperparameter tuning and final evaluation on the test set. Developed the basic methodology for MLP and Softmax. The most important lesson I learned is that sklearn `MLPClassifier` does not have drop rate choice input.

References

- [1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [2] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2009.